



QA TESTING INTERVIEW Q&A

Manual Testing + Selenium with Java

Complete Playlist-Based Interview Preparation

PART 1: MANUAL TESTING BASICS

Q1: What is Software?

Software is a set of programs, instructions, and documentation that perform specific tasks on a computer system. It's the non-physical part of computing that enables computers to perform useful functions. Software can be categorized into system software (OS, drivers) and application software (browsers, word processors, games).

Q2: Types of Software

- System Software: Operating systems, compilers, loaders
- Application Software: Microsoft Word, Google Chrome, Games
- Firmware: Embedded software in devices like routers, printers

Q3: What is Software Testing?

Software Testing is the process of evaluating a software application to identify defects or gaps against the expected behavior. It ensures the software meets functional and non-functional requirements. Testing verifies that the software works as intended and identifies bugs before release. Key aspects: finding defects, ensuring quality, validating requirements.

Q4: What is Software Quality?

Software Quality refers to the degree to which a software product meets specified requirements and user expectations. It includes functionality, reliability, usability, performance, security, and maintainability. High quality software has minimal bugs, good performance, easy-to-use interface, and meets all user requirements.

Q5: Project vs Product

- Project: Has a defined start and end date. Created for a specific client with specific requirements. Example: Developing a custom billing system for a bank
- Product: Continuous software with indefinite lifecycle. Built for a general market. Example: Microsoft Office, Google Chrome

Q6: Why do we need Testing?

- Find defects early to reduce cost of fixing
- Ensure product meets business requirements
- Build confidence in the software
- Prevent security vulnerabilities
- Ensure better user experience

Q7: What is Error, Bug, and Failure?

- Error: Mistake made by a developer in code/logic
- Bug: Result of an error that causes incorrect behavior (defect)
- Failure: When software doesn't work as expected in production

Q8: Why does software have bugs?

- Human error - developers make mistakes
- Tight deadlines - rushing code
- Complex requirements - hard to understand
- Poor communication - misunderstanding requirements
- Lack of testing - insufficient quality checks

PART 2: SDLC & TESTING CONCEPTS

Q9: What is SDLC?

SDLC (Software Development Life Cycle) is a structured process that defines the stages involved in planning, creating, testing, and deploying software. It provides a framework for organizing development activities and ensures systematic development with quality checks at each phase.

Q10: Phases of SDLC

- Planning: Feasibility study, cost analysis, resource planning
- Requirements Analysis: Gather and document what the software should do
- Design: Create architecture and design specifications
- Development/Coding: Write the actual code
- Testing: Find and report defects
- Deployment & Maintenance: Release to production and support

Q11: What is Waterfall Model?

Waterfall is a linear SDLC model where each phase flows into the next. Once a phase is completed, it's hard to go back. Requirements are frozen at the beginning. Phases: Planning → Requirements → Design → Development → Testing → Deployment. Good for projects with clear, fixed requirements. Bad for changing requirements.

Q12: What is V-Model?

V-Model (Verification and Validation) shows the relationship between development and testing phases. It's shaped like a V: Left side (development) = Planning, Requirements, Design, Development. Right side (testing) = Unit Testing, Integration Testing, System Testing, UAT. Each development phase has a corresponding testing phase.

Q13: What is Agile Model?

Agile is an iterative SDLC model that focuses on incremental delivery, customer feedback, and flexibility. Work is divided into small iterations (sprints) of 1-4 weeks. Each sprint delivers a working piece of software. Requirements can change anytime. Framework: Scrum. Good for projects with changing/unclear requirements.

Q14: Verification vs Validation

- Verification: Are we building the product RIGHT? Checks if code meets design specs. Static testing (review, inspection)
- Validation: Are we building the RIGHT product? Checks if product meets user needs. Dynamic testing (actual execution)

Q15: What is Testing Methodology?

Testing Methodology is a set of guidelines and best practices for testing software. It includes testing types, techniques, tools, and processes. Examples: Manual Testing, Automation Testing, Black Box Testing, White Box Testing. It ensures consistency, quality, and efficiency in testing.

PART 3: TESTING TYPES

Q16: What is Functional Testing?

Functional Testing verifies that the software performs the intended functions as per requirements. It tests WHAT the software does. Tests features, functionalities, buttons, forms, navigation. Black-box testing (no code knowledge). Examples: Login, Add to Cart, Submit Form.

Q17: What is Non-Functional Testing?

Non-Functional Testing verifies software quality attributes beyond basic functionality. Tests HOW the software performs. Includes: Performance, Security, Usability, Compatibility, Load, Stress testing. Examples: Is website fast? Is it secure? Is UI user-friendly?

Q18: What is Unit Testing?

Unit Testing tests individual components/modules in isolation. Done by developers. Tests smallest testable parts of code (functions, methods). White-box testing (code knowledge required). Tools: JUnit, NUnit, TestNG. Found early to prevent cascading failures.

Q19: What is Integration Testing?

Integration Testing tests interaction between integrated modules. Tests if modules work together correctly. Done after unit testing. Example: Database module + User module should pass user data correctly. Types: Top-down, Bottom-up, Big-bang.

Q20: What is System Testing?

System Testing tests the complete integrated system against requirements. Tests end-to-end workflows. Done by QA team. Includes: Functional + Non-functional testing. Example: Test entire e-commerce website (search, cart, checkout, payment).

Q21: What is Acceptance Testing?

Acceptance Testing (UAT) tests if software meets business requirements and is ready for production. Done by end-users/business analysts. Gate before release. Example: Bank verifies if new app handles all banking operations correctly.

Q22: What is Smoke Testing?

Smoke Testing verifies basic critical functionalities work without going into detail. Quick sanity check after build. Tests: Can app launch? Can user login? Can user navigate? Fails immediately if main features are broken. Done before detailed testing.

Q23: What is Sanity Testing?

Sanity Testing checks if a specific fix or small change works as expected without side effects. Narrow scope testing. Done after bug fix. Example: Fixed login button, now test only login functionality. Difference from Smoke: Sanity is targeted, Smoke is broader.

PART 4: SYSTEM TESTING TYPES

Q24: What is GUI Testing?

GUI (Graphical User Interface) Testing verifies visual elements work correctly. Tests: Buttons, Text boxes, Menus, Windows, Dialog boxes, Icons. Checks: Proper alignment, correct colors, responsive design, font sizes. Tools: Selenium, QTP. Example: Check if 'Submit' button is visible and clickable.

Q25: What is Usability Testing?

Usability Testing verifies if software is user-friendly and easy to use. Tests: Navigation, Learning curve, User satisfaction. Real users test and give feedback. Example: Can a new user understand how to use the app without help?

Q26: What is Compatibility Testing?

Compatibility Testing verifies software works on different browsers, OS, devices, screen sizes. Tests: Chrome, Firefox, Safari, IE on Windows, Mac, Linux. Mobile: iOS, Android. Example: Website should work on Chrome, Firefox, Safari.

Q27: What is Performance Testing?

Performance Testing measures software speed, responsiveness, stability under load. Checks: Load time, Response time, Throughput, Resource usage. Tools: JMeter, LoadRunner. Example: Website should load in <3 seconds with 1000 concurrent users.

Q28: What is Security Testing?

Security Testing identifies vulnerabilities and ensures data protection. Tests: SQL injection, Authentication, Authorization, Data encryption, Vulnerability scanning. Example: Can a hacker login with weak password? Is data encrypted?

PART 5: ADVANCED TESTING TYPES

Q29: What is Regression Testing?

Regression Testing verifies that new changes don't break existing functionality. Retests previously working features after updates. Example: After fixing login bug, ensure all other features still work. Automated using test suites. Time-consuming but crucial.

Q30: What is Retesting?

Retesting tests a specific bug after it's been fixed by developer. Narrow scope, only tests the exact defect reported. Example: Login button crash reported → developer fixes → tester retests login. Different from Regression: Retesting = specific fix, Regression = all features.

Q31: What is Exploratory Testing?

Exploratory Testing is unscripted, creative testing where tester explores the app like an end-user. No predefined test cases. Tester learns, tests, reports simultaneously. Finds unexpected bugs. Example: Try random features, navigation, inputs to find issues.

Q32: What is Ad-hoc Testing?

Ad-hoc Testing is random, unplanned testing without test cases. Tester improvises based on experience. Finds hidden bugs. Done when formal testing is complete. Similar to Exploratory but more random. Not repeatable/documentable.

Q33: What is Globalization Testing?

Globalization Testing verifies software works globally with different languages, currencies, date formats. Tests: Multiple languages, Character sets, Time zones. Example: Website in English, Spanish, Chinese with correct date/time format.

Q34: What is Localization Testing?

Localization Testing adapts and tests software for specific regions/languages. Tests: Translation accuracy, Cultural appropriateness, Local format. Example: Hindi version of app with Rupee currency and DD/MM/YYYY date format.

PART 6: TEST CASE DESIGN TECHNIQUES

Q35: What is Test Case?

Test Case is a detailed document specifying inputs, actions, and expected outputs for testing. Contains: Test ID, Description, Precondition, Steps, Expected Result, Actual Result, Status. Example: Test login with valid credentials → Expected: Login successful.

Q36: What is Test Scenario?

Test Scenario is a high-level description of a functionality to test. It's a user story. Example: 'User should be able to login to the system.' One scenario can have multiple test cases. Scenarios are broader than cases.

Q37: Difference between Test Case & Test Scenario

- Scenario: High-level, one liner (e.g., User Login) | Case: Detailed step-by-step (e.g., Valid username/password)
- Scenario: What to test | Case: HOW to test in detail

Q38: What is Equivalence Partitioning?

Equivalence Partitioning divides input data into groups (partitions) where all values behave the same. Tests one value from each partition. Reduces test cases. Example: Age field (0-17 invalid, 18-65 valid, 66+ valid). Test: 16 (invalid), 25 (valid), 70 (valid).

Q39: What is Boundary Value Analysis (BVA)?

BVA tests values at boundaries of input ranges. Tests edge cases. Example: Age 18-65. Test: 17 (below), 18 (boundary), 65 (boundary), 66 (above). Catches off-by-one errors. More thorough than Equivalence Partitioning.

Q40: What is Decision Table Testing?

Decision Table Testing uses a table to test combinations of inputs and conditions. Shows all possible combinations. Example: Login (valid username, valid password, remember checkbox). Tests all combinations: T-T-T, T-T-F, T-F-T, etc.

Q41: What is State Transition Testing?

State Transition Testing tests transitions between different states. Example: Order states: New → Processing → Shipped → Delivered. Tests transitions: Can order move from New to Processing? Can it skip to Delivered? Invalid transitions?

Q42: What is Error Guessing?

Error Guessing is a testing technique where tester uses experience/intuition to guess where errors might occur. No formal methodology. Based on common mistakes. Example: Entering special characters, negative numbers, NULL values. Finds unexpected bugs.

PART 7: STLC (SOFTWARE TESTING LIFE CYCLE)

Q43: What is STLC?

STLC (Software Testing Life Cycle) is a structured process that defines all activities in testing from start to end. It ensures systematic testing, proper documentation, and quality control. Parallels SDLC. Phases: Planning, Design, Execution, Closure.

Q44: Phases of STLC

1. Requirement Analysis

Understand functional/non-functional requirements. Identify testable items. Create RTM (Requirement Traceability Matrix).

2. Test Planning

Define testing scope, timeline, resources, budget. Create Test Plan. Estimate effort. Assign testers.

3. Test Design

Create test scenarios and test cases. Prepare test data. Design test strategy. Document test conditions.

4. Test Execution

Execute test cases. Log actual results. Report bugs. Retest after fixes. Document results.

5. Defect Reporting

Report bugs with details. Track defect lifecycle. Update bug status. Communicate with developers.

6. Test Closure

Complete testing activities. Generate final test report. Document lessons learned. Archive test artifacts.

PART 8: TEST DOCUMENTATION

Q45: What is Test Plan?

Test Plan is a comprehensive document that outlines the overall testing approach. Contains: Scope, Objectives, Schedule, Resources, Tools, Test Cases, Entry/Exit Criteria, Risk Analysis, Deliverables. Foundation for testing. Created before test execution.

Q46: What is Test Strategy?

Test Strategy is a high-level document that defines testing approach for organization. Contains: Types of testing, Testing methodology, Tools, Entry/Exit criteria, Risk mitigation. Applies to all projects. Test Plan is project-specific derived from Test Strategy.

Q47: What is Test Case Template?

Test Case Template is a standardized format for documenting test cases. Fields: Test ID, Test Description, Precondition, Test Steps, Expected Result, Actual Result, Status (Pass/Fail), Date, Tester Name. Ensures consistency and clarity.

Q48: What is Test Data?

Test Data is the input information used in test cases. Can be: Valid (correct), Invalid (wrong), Boundary, Positive, Negative. Sources: Production data (masked), Generated, Provided by business. Example: Username='admin', Password='123', Age=25.

Q49: What is Traceability Matrix (RTM)?

RTM (Requirements Traceability Matrix) maps requirements to test cases. Shows which requirement is covered by which test case. Ensures complete coverage. Columns: Requirement ID, Requirement Description, Test Case ID, Status. Prevents missing tests.

PART 9: DEFECT / BUG LIFECYCLE

Q50: What is Defect?

Defect is a deviation from expected behavior. It's a bug/issue found during testing. Example: Login button doesn't work, spelling error, wrong calculation. When fixed, it becomes a bug fix. Defects increase as code complexity increases.

Q51: Defect Life Cycle

- NEW: Defect just reported
- ASSIGNED: Given to developer
- IN PROGRESS: Developer is working on it

- **FIXED:** Developer claims it's fixed
- **VERIFIED/CLOSED:** Tester confirmed fix or **REOPENED:** Fix didn't work
- **DUPLICATE/REJECTED/DEFERRED:** Not a real issue or postponed

Q52: What is Severity?

Severity is how bad the defect is for the software. Levels: Critical (app crashes), Major (main feature broken), Minor (non-critical issue), Trivial (cosmetic, spelling). Defined by tester. Impact on functionality.

Q53: What is Priority?

Priority is how urgently the defect needs fixing. Levels: Critical (fix immediately), High (fix soon), Medium (fix before release), Low (can wait). Defined by manager/product owner. Business importance.

Q54: Difference between Severity & Priority

- Severity: **IMPACT** on software (high severity = major impact) | Priority: **URGENCY** to fix (high priority = fix first)
- Example: Typo in footer (low severity, low priority) vs App crashes (critical severity, critical priority)

Q55: What is Bug Report?

Bug Report documents a defect found during testing. Contains: Bug ID, Title, Description, Precondition, Steps to Reproduce, Expected vs Actual, Severity, Priority, Attachments (screenshots). Clear communication between tester and developer.

Q56: What is Defect Tracking?

Defect Tracking is the process of recording, monitoring, and managing defects throughout their lifecycle. Tools: Jira, Bugzilla, Azure DevOps. Tracks: Status, Assignment, History. Ensures no bug is missed.

PART 10: TEST EXECUTION

Q57: What is Test Execution?

Test Execution is the process of running test cases on the software. Tester follows test steps, compares actual results with expected results, documents outcomes. Logs pass/fail for each case. Raises bugs if needed.

Q58: What is Test Report?

Test Report is a document summarizing test execution results. Contains: Test summary, Tests passed/failed/skipped, Defect count, Coverage percentage, Recommendations. Shared with stakeholders. Shows readiness for release.

Q59: What is Test Metrics?

Test Metrics are quantitative measures of testing. Examples: Test case execution rate, Pass/Fail ratio, Bug count, Test coverage, Defect density, Cost per defect. Shows testing effectiveness. Data-driven decisions.

Q60: What is QA Metrics?

QA Metrics measure quality assurance efforts and product quality. Examples: Bug escape rate, Defect detection rate, Test coverage, Code quality score, Time to fix, Release quality. Shows QA effectiveness.

PART 11-16: REAL-TIME PROJECT & ADVANCED TOPICS

Q61: What is Test Plan in real project?

In real projects, Test Plan includes: Project overview, Testing scope, Schedule (with deadlines), Resources (QA lead, testers, automation), Tools (Jira, Selenium), Test cases count, Budget, Risks, Approval sign-offs. Created at project start after requirements finalization.

Q62: How to write Test Scenarios?

Test Scenarios should be user-focused. Format: 'User should be able to [action] by [steps]'. Examples: 'User should be able to login with valid credentials.' 'User should not be able to login with invalid password.' Based on requirements. One-liner, high-level. Multiple test cases derive from one scenario.

Q63: How to write Test Cases?

Test Cases should be detailed and clear. Include: Test ID, Title, Description, Precondition, Steps (numbered, clear), Expected Result (specific), Actual Result (filled during execution), Status. Example: Login Test Case with steps: 1. Open app, 2. Click login, 3. Enter valid username, 4. Enter valid password, 5. Click submit, 6. Expected: Dashboard opens.

Q64: What is Environment Setup?

Environment Setup is preparing the testing infrastructure. Includes: Setting up test servers, databases, installing browsers, tools (Selenium, Jira), configuring network, preparing test data. Must match production. Quality of setup affects test reliability.

Q65: What is Build Deployment?

Build Deployment is the process of releasing a built version of software to testing environment. Developer compiles code → Build created → Deployed to QA environment. Example: Build 1.2.3 deployed to QA server. Testers then execute tests on this build.

Q66: Smoke vs Sanity Testing

- SMOKE: Broad quick check of main features. First testing after build. If fails, reject build.
- SANITY: Targeted testing of specific fix/change. Narrow scope. After bug fix. Checks no side effects.

Q67: What is RTM and why important?

RTM maps each requirement to test cases. Ensures complete coverage. Shows: Which test case covers which requirement. Why important: Prevents missing requirements, tracks coverage percentage, ensures quality, auditable for compliance. Example: Requirement 'User login' → Test cases: Valid login, Invalid password, Blank fields.

Q68: Agile Methodology

Agile is iterative development approach. Focus: Incremental delivery, Customer feedback, Flexibility. Development and testing happen in parallel. Short sprints (2-4 weeks). Requirements can change anytime. Continuous integration and testing. vs Waterfall: Flexible, Adaptive, Faster feedback.

Q69: Scrum Framework

Scrum is an Agile framework with defined roles and ceremonies. Roles: Product Owner (requirements), Scrum Master (facilitator), Team (developers + testers). Events: Sprint Planning, Daily Standup, Sprint Review, Retrospective. Deliverable: Potentially shippable product increment after each sprint.

Q70: Scrum Concepts

- Sprint: 2-4 week iteration
- Sprint Planning: Team chooses work for sprint
- Daily Standup: 15-min sync on progress
- Sprint Review: Demo completed work
- Retrospective: Team reflects on improvements

Q71: Jira Tool

Jira is a project/issue tracking tool. Used for: Bug tracking, Sprint planning, Task management, Release management. Key features: Create issues, Track status, Link related items, Generate reports, Dashboards. Widely used in Agile teams.

Q72: How to create bug in Jira?

Steps: 1. Click 'Create' button, 2. Select Issue Type = Bug, 3. Fill Project, 4. Enter Summary (bug title), 5. Enter Description (details), 6. Add Severity/Priority, 7. Upload screenshot, 8. Assign to developer, 9. Click Create. Bug is created and assigned.

Q73: Workflow in Jira & Ticket Lifecycle

- Workflow: Open → In Progress → In Review → Done
- Bug Workflow: New → Assigned → Fixed → Verified → Closed
- Transition: Click button to move from one status to next

PART 17: REAL-TIME INTERVIEW QUESTIONS (VERY IMPORTANT)

Q74: Explain STLC with example

STLC = Software Testing Life Cycle. Phases: (1) Requirement Analysis: Understand app features to test. (2) Test Planning: Define schedule, resources, tools. (3) Test Design: Create 50+ test cases. (4) Test Execution: Run test cases, log results. (5) Defect Reporting: Report 10 bugs. (6) Test Closure: Generate report. Example: Testing e-commerce website login feature through all 6 phases until app is ready for production.

Q75: Difference between STLC & SDLC

- SDLC: Overall software development (includes planning, design, coding, testing, deployment)
- STLC: Only testing part of SDLC

- SDLC: Done by entire team | STLC: Done by QA team
- SDLC: 6-7 phases | STLC: 6 phases within SDLC

Q76: What is Bug Life Cycle?

Bug Lifecycle: Tester finds bug → Reports (NEW) → Manager assigns to developer (ASSIGNED) → Developer works (IN PROGRESS) → Marks FIXED → Tester retests → Bug closes (CLOSED) OR Tester finds fix didn't work (REOPENED). Status can be: New, Assigned, In Progress, Fixed, Verified, Closed, Reopened, Deferred, Duplicate.

Q77: Severity vs Priority practical example

- Example 1: Typo in footer → Low severity (cosmetic) + Low priority (can wait)
- Example 2: Payment button doesn't work → Critical severity (core feature broken) + Critical priority (fix immediately)
- Example 3: Spell error in welcome message → Low severity but High priority (client complaint)

Q78: Write test cases for login page

Test Case 1: Valid Credentials Steps: 1. Open login page 2. Enter username='admin' 3. Enter password='123' 4. Click Login Expected: Dashboard displayed Test Case 2: Invalid Username Steps: 1. Open login page 2. Enter username='invalid' 3. Enter password='123' 4. Click Login Expected: Error message shown Test Case 3: Empty Fields Steps: 1. Open login page 2. Leave username empty 3. Leave password empty 4. Click Login Expected: Error message "Please fill all fields" Test Case 4: Forgot Password Steps: 1. Open login page 2. Click 'Forgot Password' 3. Enter email 4. Click Submit Expected: Reset email sent Test Case 5: Remember Me checkbox Steps: 1. Enter valid credentials 2. Check 'Remember Me' 3. Login Expected: Next time browser pre-fills username

Q79: Explain Agile process

Agile Process: Sprint starts with Planning meeting → Team picks user stories → Developers code + Testers test in parallel during sprint (2 weeks) → Daily standup (what done/todo/blockers) → Sprint ends with Review (demo to client) → Retrospective (team improvement discussion) → Repeat. Advantages: Fast feedback, flexibility, quality, team collaboration. Challenges: Scope creep, resource availability.

Q80: How do you handle defects in project?

Process: (1) Find defect during testing (2) Create detailed bug report in Jira with steps to reproduce (3) Set severity/priority (4) Assign to developer (5) Communicate (email/chat) (6) Developer fixes (7) Tester retests (8) Verify fix works without side effects (9) Close bug. If reopened, repeat process. Track: Time to fix, bug count, escape rate.

SELENIUM WITH JAVA – INTERVIEW Q&A

Q81: What is Selenium?

Selenium is an open-source automation testing tool for web applications. Enables automated testing across browsers and platforms. Supports multiple programming languages (Java, Python, C#, Ruby). Components: WebDriver, Grid, IDE. Reduces manual testing time. Free and widely used in industry.

Q82: Components of Selenium

- Selenium IDE: Record and playback tool (no coding)
- Selenium WebDriver: API for automation (most used)
- Selenium Grid: Parallel testing on multiple browsers/machines

Q83: What is WebDriver?

WebDriver is the main component of Selenium. It's a programming interface that controls a web browser. Communicates with browser drivers (ChromeDriver, FirefoxDriver). Example: WebDriver driver = new ChromeDriver();

Q84: Advantages of Selenium WebDriver

- Supports multiple languages (Java, Python, C#)
- Cross-browser compatibility
- Fast execution
- Free and open-source
- Good community support

Q85: Disadvantages of Selenium

- Cannot test desktop applications
- Cannot test mobile apps directly
- No built-in reporting (need frameworks)
- Difficult learning curve
- Performance depends on code quality

Q86: Selenium WebDriver Architecture

Architecture: Test Script (Java) → Selenium WebDriver (Client library) → JSON Wire Protocol → Browser Driver (ChromeDriver/FirefoxDriver) → Browser (Chrome/Firefox). Client sends commands via protocol → Driver communicates with browser → Executes actions.

Q87: get() vs navigate().to()

- get(): Waits for page load. Example: driver.get("https://google.com");
- navigate().to(): Doesn't wait. Example: driver.navigate().to("https://google.com");

Q88: How to get page title?

```
String title = driver.getTitle(); System.out.println(title);
```

Q89: How to get current URL?

```
String currentUrl = driver.getCurrentUrl();  
System.out.println(currentUrl);
```

Q90: How to get page source?

```
String pageSource = driver.getPageSource();  
System.out.println(pageSource);
```

Q91: What is WebElement?

WebElement is an interface representing an HTML element. Examples: Button, Text box, Link, Checkbox. Methods: click(), sendKeys(), getText(), isDisplayed(), isEnabled(). Example: `WebElement loginBtn = driver.findElement(By.id("login"));`

Q92: findElement() vs findElements()

- `findElement()`: Returns first matching element. Throws `NoSuchElementException` if not found. Returns `WebElement`.
- `findElements()`: Returns list of all matching elements. Returns empty list if not found. No exception thrown. Returns `List<WebElement>`.

Q93: How to send text?

```
WebElement username = driver.findElement(By.id("username"));
username.sendKeys("admin");
```

Q94: How to clear text?

```
WebElement textField = driver.findElement(By.id("email"));
textField.clear();
```

Q95: getText() vs getAttribute()

- `getText()`: Gets visible text of element. Example: `<button>Click Me</button>` → `getText() = "Click Me"`
- `getAttribute()`: Gets attribute value. Example: `<input value="hello">` → `getAttribute("value") = "hello"`

Q96: How to handle dropdown?

```
Select dropdown = new Select(driver.findElement(By.id("country")));
dropdown.selectByValue("USA"); // or selectByVisibleText or
selectByIndex
```

Q97: How to select checkbox?

```
WebElement checkbox = driver.findElement(By.id("agree")); if (!
checkbox.isSelected()) { checkbox.click(); }
```

Q98: How to switch windows?

```
Set<String> windows = driver.getWindowHandles(); for (String window :
windows) { driver.switchTo().window(window); }
```

Q99: What is broken link?

Broken link is a link that doesn't lead to valid page. Returns HTTP status codes 4xx or 5xx. Example: href points to deleted page. To check: Get URL from href → Use `HttpURLConnection` → Check response code (200 = valid, 404 = broken).

Q100: How to handle alerts?

```
Alert alert = driver.switchTo().alert(); alert.accept(); // OK button  
alert.dismiss(); // Cancel button alert.sendKeys("text"); // Type in  
alert alert.getText(); // Get alert text
```

Q101: What is iframe?

iframe is inline frame embedding another webpage. Example: Maps in website. To interact:
driver.switchTo().frame(0) or frame("frameName") or frame(webelement). Then perform actions.
Switch back: driver.switchTo().defaultContent();

Q102: What is synchronization problem?

Synchronization problem occurs when script runs faster than application response. Example: Click button before page fully loads. Solution: Use Waits. Implicit Wait: Applied to all elements. Explicit Wait: Applied to specific element with conditions.

Q103: Implicit vs Explicit wait

- Implicit Wait: driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS); Applies to all elements. Waits up to 10s.
- Explicit Wait: WebDriverWait wait = new WebDriverWait(driver, 10);
wait.until(ExpectedConditions.elementToBeClickable(By.id("btn"))); Specific element. Better control.

Q104: How to handle web table?

```
List<WebElement> rows = driver.findElements(By.xpath("//table//tr"));  
for (WebElement row : rows) {    String data = row.getText(); }
```

Q105: What is Actions class?

Actions class provides methods for advanced interactions. Examples: hover, right-click, double-click, drag-drop. Usage: Actions actions = new Actions(driver); actions.moveToElement(element).perform();

Q106: How to perform mouse hover?

```
Actions actions = new Actions(driver);  
actions.moveToElement(driver.findElement(By.id("menu"))).perform();
```

Q107: How to capture screenshot?

```
TakesScreenshot screenshot = (TakesScreenshot) driver; File srcFile =  
screenshot.getScreenshotAs(OutputType.FILE);  
FileUtils.copyFile(srcFile, new File("screenshot.png"));
```

Q108: What is JavaScriptExecutor?

JavaScriptExecutor executes JavaScript code in browser. Used when Selenium methods don't work. Example: Click hidden element, Scroll page. Usage: JavaScriptExecutor js = (JavaScriptExecutor) driver; js.executeScript("alert('Hello')");

Q109: How to scroll using JS?

```
JavaScriptExecutor js = (JavaScriptExecutor) driver;  
js.executeScript("window.scrollTo(0, 500)"); // Scroll down 500px
```

Q110: What are cookies?

Cookies are small files stored in browser. Used to remember user data (login, preferences). To add:
`driver.manage().addCookie(new Cookie("name", "value"));` To delete:
`driver.manage().deleteCookie("name");`

Q111: How to upload file?

```
WebElement uploadButton = driver.findElement(By.id("fileUpload"));  
uploadButton.sendKeys("C://Users//file.txt");
```

Q112: How to download file?

Set download path in ChromeOptions before creating driver. Example: `ChromeOptions options = new ChromeOptions(); options.addArguments("download.default_directory=C://Downloads");` Then click download link.

Q113: How to connect database in Selenium?

```
Class.forName("com.mysql.jdbc.Driver"); Connection conn =  
DriverManager.getConnection("jdbc:mysql://localhost:3306/db", "user",  
"pass"); Statement stmt = conn.createStatement(); ResultSet rs =  
stmt.executeQuery("SELECT * FROM users");
```

Q114: Selenium best practices

- Use Page Object Model (POM) for maintainability
- Use explicit waits instead of implicit
- Handle exceptions properly
- Use DataDriven testing
- Close browser after test
- Use version control (Git)

INTERVIEW TIPS & FINAL TIPS

Before Interview:

- Practice all Q&A questions by heart
- Prepare real project examples to explain
- Know your resume details
- Practice speaking clearly and confidently
- Have questions ready to ask interviewer

During Interview:

- Listen carefully and answer fully
- Give examples from your experience
- Admit if you don't know something, don't bluff

- Ask for clarification if unclear
- Be positive and enthusiastic

Expected Salary Ranges (India - 2024):

- QA Fresher (0 yrs): 2-3 LPA
- QA Associate (1-2 yrs): 3-5 LPA
- QA Engineer (2-4 yrs): 5-8 LPA
- Senior QA (4-7 yrs): 8-15 LPA
- QA Lead (7+ yrs): 15+ LPA

 **You are now READY for QA interviews!** 